

Task 1: Deadlock detection using a resource allocation graph for a single quantity of each resource using a cycle detection algorithm.

```
#include <stdio.h>
#define MAX 26

typedef struct {
    int adj[MAX][MAX];
} Graph;

int dfs(Graph* g, int v, int visited[], int recStack[]) {
    visited[v] = 1;
    recStack[v] = 1;
    for (int i = 0; i < MAX; i++) {
        if (g->adj[v][i]) {
            if (!visited[i] && dfs(g, i, visited, recStack))
                return 1;
            else if (recStack[i])
                return 1;
        }
    }
    recStack[v] = 0;
    return 0;
}

int deadlockCheck(Graph* g) {
    int visited[MAX] = {0};
    int recStack[MAX] = {0};
    for (int i = 0; i < MAX; i++) {
        if (!visited[i]) {
            if (dfs(g, i, visited, recStack))
                return 1;
        }
    }
    return 0;
}

int main() {
    Graph g;
    int e,n;
    for (int i = 0; i < MAX; i++)
        for (int j = 0; j < MAX; j++)
            g.adj[i][j] = 0;
    printf("Number of Nodes: ");
}
```

```

scanf("%d", &n);
printf("Number of Edges: ");
scanf("%d", &e);
printf("Edge:\n");
for (int i = 0; i < e; i++) {
    char fromChar, toChar;
    scanf(" %c %c", &fromChar, &toChar);
    int from = (fromChar >= 'A' && fromChar <= 'Z') ? fromChar - 'A' :
        (fromChar >= 'a' && fromChar <= 'z') ? fromChar - 'a' : -1;
    int to = (toChar >= 'A' && toChar <= 'Z') ? toChar - 'A' :
        (toChar >= 'a' && toChar <= 'z') ? toChar - 'a' : -1;
    if (from >= 0 && to >= 0)
        g.adj[from][to] = 1;
}
printf("Deadlock: ");
if (deadlockCheck(&g))
    printf("Yes\n");
else
    printf("No\n");
return 0;
}

```

Input:

Number of Nodes: 13

Number of Edges: 13

Edges:

R A

A S

C S

F S

W F

D S

U D

G U

V G

D T

T E

E V

B T

Output:

Deadlock: Yes